ENTERPRISE CI/CD PIPELINE ARCHITECTURE (AZURE DEVOPS & GITHUB ACTIONS)

End-to-End Delivery Lifecycle | YAML Multi-Stage Pipelines | Branch Policies | Automated Build, Test & Deployment to Azure Linux & Storage

CI/CD PIPELINE ARCHITECTURE
Osama Khalil
Senior DevOps Engineer | CI/CD & Automation Specialist

PHASE 1: SOURCE CONTROL & BRANCH GOVERNANCE

1. Developer Workstation (Inner Loop)
- Enterprise Linux / macOS / VS Code environment
- Python FastAPI development & Telecom UI updates
- Local unit test runs (pytest) & lint checks (flake8/black)

2. Git Feature Branch (feat/nms-api-v2)
- Branched strictly from latest main branch
- Standardized atomic commits with clear messages
- Pushed to remote Azure Repos / GitHub repository

3. Pull Request (PR) Initiated
- PR opened targeting protected main branch
- Mandatory linkage to JIRA user story / defect ticket
- Automated PR validation pipeline triggered immediately

4. Code Review & Branch Policies Gate
- [PASS] Minimum 2 peer code reviews approved
- [PASS] Automated CI build & validation pipeline passed
- [PASS] All review comments & threads resolved
- [PASS] Direct push to main branch strictly prohibited

Merge to main Branch (Trunk)
Fast-forward / squash merge kicks off deployment pipeline

Triggers CI

PHASE 2: CONTINUOUS INTEGRATION (CI PIPELINE)

5. CI/CD Orchestrator (YAML Pipelines)
Azure DevOps Pipelines / GitHub Actions
Configuration as Code: azure-pipelines.yml

STAGE 1: CODE QUALITY & LINTING
- Python Flake8 & Black: PEP8 & syntax style
- yamllint: validate pipeline & infrastructure configs
- ShellCheck: enterprise Bash deployment scripts

STAGE 2: AUTOMATED TESTING & BUILD
- pytest suite for FastAPI telecom API endpoints
- Mock database integration tests with PostgreSQL
- Code coverage metrics & test results publication

STAGE 3: SHIFT-LEFT SECURITY
- Checkov & tfsec: scan Terraform IaC templates
- Detect open NSG ports & unencrypted resources
- Secret scanner: guarantee zero hardcoded secrets

STAGE 4: ARTIFACT GENERATION
- Compile and package frontend SPA static files
- Package FastAPI app, requirements & systemd units
- Publish verified drop to Azure Pipeline Artifacts

CI Output: Verified & Immutable Artifact Drop
Triggers Continuous Deployment release pipeline to Azure

Artifact Drop

PHASE 3: CONTINUOUS DEPLOYMENT (CD) TO AZURE

7. CD Release Pipeline (Multi-Stage Promotion)
Automated Azure Service Connection (SPN) with Scoped RBAC
Zero shared credentials; service principal with least privilege

ENVIRONMENT 1: DEV (AUTOMATED CONTINUOUS DEPLOY)
- Automatic trigger upon successful CI build drop
- Sync frontend static files to Azure Storage $web container
- Deploy backend code to Linux VM via Bash script & systemd restart
- Automated smoke tests: health endpoint probe (/healthz)

ENVIRONMENT 2: STAGE (PRE-PRODUCTION VALIDATION)
- Automated deployment to isolated staging Azure subscription
- PostgreSQL schema migration dry run & data integrity check
- End-to-end integration test suite against staging services
- Performance benchmarking under simulated network traffic

Production Approval Gate (Manual Sign-off)
Mandatory sign-off by Lead DevOps & Operations before release

ENVIRONMENT 3: PRODUCTION AZURE ENVIRONMENT
- Frontend synced to live Azure Storage Static Website with CDN
- Zero-downtime rolling reload on Linux VM (Nginx + FastAPI)
- Private connectivity to Azure PostgreSQL Flexible Server
- Prometheus & Azure Monitor active telemetry verification

[ACTIVE] Enterprise Production Status: HEALTHY & OPERATIONAL

Technologies:
Azure DevOps | GitHub Actions | YAML | Git | Terraform | Linux | Nginx | FastAPI | PostgreSQL